HOSPITAL MANAGEMENT SYSTEM

STUDENT PROJECT ASSIGNMENT DOCUMENT

Project Title: Hospital Management System

Difficulty Level: Intermediate to Advanced

Technology: Python

Concepts Covered:

- Lists
- Dictionaries
- Sets
- Functions
- Object-Oriented Programming
- Abstraction
- Inheritance
- Encapsulation
- Polymorphism
- Custom Exception Handling
- Menu Driven Applications

Submission Type: Individual Assignment

PROJECT BACKGROUND

A hospital currently manages patient and doctor records manually using paper files and spreadsheets.

The management team wants a Hospital Management System that can:

- Store doctor information
- Store patient information
- Book appointments
- Track treatment history
- Generate bills
- Produce reports

You have been assigned as a software developer to design and develop this system using Python.

PROJECT OBJECTIVE

Develop a console-based Hospital Management System that follows Object-Oriented Programming principles and implements proper data management and exception handling.

The application should be modular, maintainable, and scalable.

BUSINESS REQUIREMENTS

The hospital should be able to:

1. Add doctors
2. Add patients
3. Search doctors
4. Search patients
5. View all doctors
6. View all patients
7. Book appointments
8. View appointments
9. Add treatment records
10. View treatment history
11. Generate bills
12. Discharge patients
13. Generate reports

MANDATORY OOP REQUIREMENTS

Students must implement all four pillars of OOP.

---

1. ABSTRACTION

Create an abstract class named Person.

Attributes:

id name age phone

Abstract Method:

display_details()

The Person class should not be instantiated directly.

---

1. INHERITANCE

Create the following classes:

Doctor Patient

Both classes must inherit from Person.

Doctor Attributes:

doctor_id specialization consultation_fee

Patient Attributes:

patient_id disease admission_status

---

1. ENCAPSULATION

Make the following attributes private:

Doctor

__consultation_fee

Patient

__medical_history

Access them using:

Getter Methods Setter Methods

Examples:

get_consultation_fee() set_consultation_fee()

get_medical_history() add_medical_history()

---

1. POLYMORPHISM

Implement display_details() differently in:

Doctor Class

Patient Class

Doctor Example Output:

Doctor ID: D101 Name: Ravi Specialization: Cardiology Consultation Fee: 1000

Patient Example Output:

Patient ID: P101 Name: Kumar Disease: Fever Status: Admitted

COLLECTION REQUIREMENTS

LIST

Use a List to store appointments.

Example:

appointments = []

Each appointment should contain:

appointment_id doctor_id patient_id date time

DICTIONARY

Use Dictionary to store doctors.

Example:

doctors = { "D101": doctor_object }

Use Dictionary to store patients.

Example:

patients = { "P101": patient_object }

SET

Use Set to store unique diseases.

Example:

diseases = { "Fever", "Diabetes", "Cancer" }

Duplicate diseases should not be stored.

CUSTOM EXCEPTION REQUIREMENTS

Create the following custom exceptions.

DoctorNotFoundException

Raised when a doctor does not exist.

PatientNotFoundException

Raised when a patient does not exist.

AppointmentAlreadyExistsException

Raised when an appointment slot is already booked.

InvalidAgeException

Raised when age is invalid.

InvalidFeeException

Raised when consultation fee is invalid.

BillingException

Raised when billing calculation fails.

FUNCTIONAL REQUIREMENTS

FEATURE 1

ADD DOCTOR

Input:

Doctor ID Name Age Phone Number Specialization Consultation Fee

Validation:

Doctor ID must be unique. Age must be greater than zero. Consultation fee must be greater than zero.

Output:

Doctor added successfully.

FEATURE 2

ADD PATIENT

Input:

Patient ID Name Age Phone Number Disease

Validation:

Patient ID must be unique. Age must be greater than zero.

Output:

Patient added successfully.

FEATURE 3

VIEW ALL DOCTORS

Display:

Doctor ID Name Specialization Consultation Fee

FEATURE 4

VIEW ALL PATIENTS

Display:

Patient ID Name Disease Admission Status

FEATURE 5

SEARCH DOCTOR

Input:

Doctor ID

Output:

Doctor Details

Exception:

DoctorNotFoundException

FEATURE 6

SEARCH PATIENT

Input:

Patient ID

Output:

Patient Details

Exception:

PatientNotFoundException

FEATURE 7

BOOK APPOINTMENT

Input:

Appointment ID Doctor ID Patient ID Date Time

Validation:

Doctor exists. Patient exists. Appointment slot not already booked.

Output:

Appointment booked successfully.

Exception:

AppointmentAlreadyExistsException

FEATURE 8

VIEW APPOINTMENTS

Display:

Appointment ID Doctor Name Patient Name Date Time

FEATURE 9

ADD TREATMENT RECORD

Input:

Patient ID Treatment Description

Example:

Blood Test Diabetes Checkup Fever Treatment

Store treatment history inside Patient object.

FEATURE 10

VIEW TREATMENT HISTORY

Input:

Patient ID

Output:

Complete treatment history.

FEATURE 11

GENERATE BILL

Input:

Patient ID Medicine Charges Lab Charges

Logic:

Total Bill = Consultation Fee + Medicine Charges + Lab Charges

Output:

Billing Summary

Example:

Consultation Fee : 1000 Medicine Charges : 500 Lab Charges : 700

Total Bill : 2200

FEATURE 12

DISCHARGE PATIENT

Input:

Patient ID

Process:

Update patient status from

Admitted

to

Discharged

Output:

Patient discharged successfully.

FEATURE 13

GENERATE REPORTS

Report 1

Total Doctors

Report 2

Total Patients

Report 3

Total Appointments

Report 4

Unique Diseases

Report 5

Total Revenue Generated

MENU REQUIREMENTS

Display the following menu repeatedly until Exit is selected.

1. Add Doctor
2. Add Patient
3. View Doctors
4. View Patients
5. Search Doctor
6. Search Patient
7. Book Appointment
8. View Appointments
9. Add Treatment
10. View Treatment History

11. Generate Bill

12. Discharge Patient

13. Reports

14. Exit

SAMPLE TEST DATA

Doctors

D101 Ravi 45 9876543210 Cardiology 1000

D102 Suresh 50 9876543211 Neurology 1500

Patients

P101 Kumar 30 9876543222 Fever

P102 Ramesh 45 9876543223 Diabetes

Appointments

A101 D101 P101 10-06-2026 10:00 AM

A102 D102 P102 10-06-2026 11:00 AM

DELIVERABLES

Students must submit:

1. Complete Source Code

2. Screenshots of Execution

3. Class Diagram

4. Exception Handling Demonstration

5. Sample Output Document

6. Project Explanation Document

EVALUATION CRITERIA

OOP Implementation - 25 Marks

Collections Usage - 15 Marks

Exception Handling - 15 Marks

Business Logic - 20 Marks

Code Quality - 10 Marks

Menu Driven Functionality - 10 Marks

Documentation - 5 Marks

Total - 100 Marks

BONUS REQUIREMENTS

Additional marks will be awarded for:

1. Data Persistence using JSON
2. CSV Report Export
3. Doctor Availability Tracking
4. Appointment Cancellation
5. Search by Disease
6. Revenue Analytics
7. Sorting Doctors by Fee
8. Sorting Patients by Age
9. Multi-Hospital Support
10. Unit Testing

END OF ASSIGNMENT